

M7 — Scenes engine (palette + inject)

Phase: 4 — Scenes **Status:** DONE (2026-05-26) **Blocked by:** M4 **Blocks:** none (independent feature)

Estimated effort: 2 days · **Actual:** 0.5 day (the route surface from M4 carried the contract; M7 swapped the stub `_apply_scene` for a dispatching palette)

1. Purpose

Replace M4's stub `_apply_scene` (which wrote a single `instructor.trigger` event regardless of scene kind) with a templated palette of 8 built-in scene kinds that emit the right event types. Each scene leaves a properly-typed footprint that the EHR `fold()` projection already understands — so chart-rendering, debrief aggregation (M14), and existing v6 students see the scene's effect as native chart activity, not as a synthetic instructor event.

The forward-compatibility contract: unknown scene kinds keep working, falling back to a single `instructor.trigger`. Adding a new built-in kind later doesn't break clients that already emit it.

2. Structure

New file:

- `portal/scenes.py` — palette (`PALETTE, palette()`) plus the `apply(enc, scene, by=...)` dispatcher and 8 per-kind handlers.

Files touched:

- `portal/server.py` — imports `scenes`; `_apply_scene` becomes a one-line delegate to `scenes.apply(enc, scene, by=by)`; adds `GET /api/scenes/palette` endpoint.
- `tests/v7/test_room_api.py` — updates one assertion in `test_api_scene_broadcast_writes_chart_event_per_target` to reflect that scene broadcasts now write the **typed** event (`vitals.record`) instead of the M4 stub `instructor.trigger`.

3. Uses

- The M4 routes `POST /api/encounter/{id}/scene` and `POST /api/room/scene_broadcast` already call `_apply_scene`; after M7 those calls land in `scenes.apply`. No client change needed — the existing M5 dashboard scene-injector dialog and any direct HTTP client emits the right events transparently.
- `GET /api/scenes/palette` exposes the 8 built-ins so the dashboard scene dialog can eventually fetch them server-side rather than hard-coding the list in `control_room.js`.
- M11/M12 (Activity catalog) will store scenes as a sequence inside an Activity record; the instructor fires them in the live room.
- M14 (cohort debrief) will filter scene-emitted events out of student activity stats via the source: `'scene'` payload tag.

4. Functions (exported API surface)

Module-level

Symbol	Signature	Purpose
PALETTE	list[dict]	The 8 built-in scene entries — each {kind, label, category, default_params, description}.
palette()	() -> list[dict]	Returns a copy of PALETTE for the /api/scenes/palette route.
apply(enc, scene, *, by="instructor")	(Encounter, {kind?, params?}, by) -> dict	Public entry point. Dispatches on scene['kind'] to one of the handlers; unknown kinds route to _handle_default. Returns {ok, kind, encounter_id, category, ...}.

Scene kinds

Kind	Category	Events emitted
vitals.drop	chart	1 × vitals.record (hypotensive preset, overridable)
vitals.rise	chart	1 × vitals.record (sympathetic-surge preset, overridable)
lab.result	chart	1 × result.acknowledge with {panel, values}
order.new	chart	1 × order.place (instructor-authored MD order)
family.arrives	chart	1 × note.save (communication / consent note)
pump.alarm	device or chart_fallback	1 × device alarm.injected when an IV / enteral pump is bound; else 1 × chart instructor.trigger
code.blue	compound	1 × vitals.record (crash) + 1 × note.save (CODE BLUE) + 1 × instructor.trigger (marker) + (optional) 1 × device alarm.injected if a pump is bound
note.instructor	chart	1 × note.save (free-form text)
unknown	chart_fallback	1 × instructor.trigger carrying the raw scene payload

Payload tagging

Every event a scene writes carries:

```
{"source": "scene", "scene_kind": "<kind>", "by": "<who fired it>", ...}
```

Compound scenes additionally tag each child event with compound_role: 'crash_vitals' | 'code_announcement' | 'marker' | 'pump_alarm'.

New HTTP route

Method	Path	Auth	Returns
GET	/api/scenes/palette	operator vault	{"palette": [<8 entries>]}

5. Limitations

- **No autonomous physiology yet.** A `vitals.drop` writes one

vitals row at fire time; the subsequent vitals trajectory is still instructor-driven. Autonomous physiology is a v7.1 candidate (P6 §10).

- **Pump alarm picks the first bound pump.** If an encounter has

multiple pumps (rare in practice), only the first iterates. A follow-up could let the scene specify `station_id` in params.

- **No state guards.** The handlers don't check `enc.state` — a

scene fires even on a paused or ended encounter (the `chart_event` still records). M4's route layer is the right place to enforce state guards if needed.

- **No scheduled scenes / sequences.** The instructor fires each

scene manually. M11/M12 Activity authoring will record a **sequence** of scenes, but each fires only on operator command.

- **No undo.** `chart_event` is append-only — a fired scene can't be

retracted, only annotated. This is consistent with v6's append-only chart contract.

- **Order.new uses synthetic order_ids** prefixed `scene-ord-`. M14

debrief may want to distinguish scene-authored orders from student-authored ones; the `source: scene` tag handles that.

6. Test status

Automated (tests/v7/test_scene_*.py)

Test file	Cases	Status	Last run
test_scene_vitals_drop_writes_vitals_record.py	3 — drop defaults, param overrides, vitals.rise sister scene	PASS	2026-05-26
test_scene_pump_alarm_emits_device_event_when_bound.py	2 — bound pump → device_event + isolation, cabinet bound → fallback	PASS	2026-05-26

Test file	Cases	Status	Last run
test_scene_code_blue_compound_emits_expected_events.py	4 — no pump (3 chart events), with pump (3 chart + 1 device), room broadcast (3×N), palette endpoint	PASS	2026-05-26

9/9 PASS. **Full v7 suite: 36/36 passing** (up from 27). **Full v6 regression on v7: 147 passed**, same 6 env-flaky pre-existing failures, **0 v7 regressions**.

Manual (browser + curl — 2026-05-26)

Flow	Result
GET /api/scenes/palette after login	PASS — returns 8 entries with the expected kinds.
POST /api/encounter/{id}/scene with code.blue via direct fetch	PASS — {category: "compound", event_ids: [3 ids], device_event_id: null}.
/api/room/state immediately after firing	PASS — target encounter's chart_event_count jumped from 0 → 3, sibling stayed at 0.

7. Change list

Date	Author	Change	Files
2026-05-26	claude-cod	New portal/scenes.py with PALETTE + apply() + 8 per-kind handlers + unknown-kind fallback. _apply_scene in server.py delegates. GET /api/scenes/palette endpoint added. M4 test assertion updated to reflect typed event emission. 9 acceptance tests across 3 files.	portal/scenes.py, portal/server.py, tests/v7/test_room_api.py, tests/v7/test_scene_vitals_drop_writes_vitals_record.py, tests/v7/test_scene_pump_alarm_emits_device_event_when_bounded.py, tests/v7/test_scene_code_blue_compound_emits_expected_events.py

8. Open questions / known issues

- The M5 dashboard's scene-injector dialog still hard-codes the

7-kind list in JS. Follow-up: have control_room.js fetch /api/scenes/palette on dialog open so the palette is server-authoritative. (Low priority — keeping both in sync is one PR line each side.)

- _handle_order_new defaults patient_id to "". The v6

comparison engine's order-matching uses patient_id as a join key; if a scene-authored order needs to match a known patient, the scene's params must supply the patient id. M14 cohort debrief can highlight scene-orders separately via the source: scene tag.

- `_find_pump_station` iterates `enc.device_stations.values()` in

insertion order. If the operator binds multiple pumps to one encounter, the alarm targets whichever was bound first. This matches v6 device-routing behavior but deserves explicit documentation in the operator guide.

- The forward-compatible fallback writes one `instructor.trigger` per

unknown kind. A future iteration could add a `validate_palette` warning if the operator types a typo in the dashboard dialog — currently they get a silently-recorded fallback event.

- Scene payloads can grow large (`lab.result` with many values, `code.blue`

compound markers). The `chart_event` payload `_json` column is unbounded TEXT so no immediate storage concern, but cohort-debrief render performance should be measured in M14/M15 with realistic scene density.